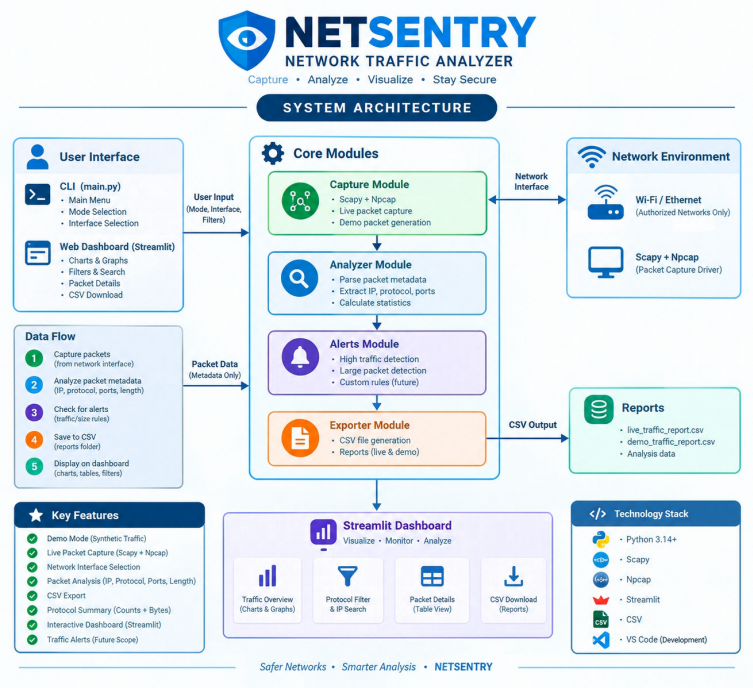
NETSENTRY

NETWORK TRAFFIC ANALYZER

FINAL PROJECT REPORT

A Python-based application for network packet metadata capture, protocol analysis, traffic statistics, alerts, and CSV reporting.



Safety principle: Use only on networks you own or are explicitly authorized to monitor. NETSENTRY focuses on packet metadata and does not display payloads, passwords, or private messages.

Prepared for academic and portfolio use

Contents

1. Abstract
2. Problem Statement and Objectives
3. Technologies Used
4. System Architecture
5. Features and Workflow
6. Project Structure
7. Testing and Results
8. Security, Limitations, and Future Work
9. Conclusion and References

Report formatting note: Text, headings, tables, and page elements are rendered as vector PDF content for sharp viewing and printing. The architecture graphic is embedded at its original image resolution.

1. Abstract

NETSENTRY is a Python-based network traffic analyzer designed to demonstrate essential networking and cybersecurity concepts. It provides a Demo Mode that uses synthetic TCP, UDP, and ICMP records, and a Live Mode that captures a bounded number of packets from a selected interface using Scapy and Npcap. The application extracts metadata, calculates traffic statistics, applies basic alert rules, exports CSV reports, and provides a Streamlit dashboard for visual analysis.

2. Problem Statement and Objectives

Understanding network traffic is an important foundation for networking and cybersecurity. Beginners need a controlled way to observe packet metadata, identify protocols, review traffic statistics, and practice reporting without inspecting private payload content. NETSENTRY addresses this learning need through a small modular application with demo data, bounded live capture, automated tests, and visual reporting.

• Provide a Demo Mode using synthetic TCP, UDP, and ICMP records.

• Capture a limited number of packets from a selected interface on an authorized network.

• Extract timestamp, source and destination IP addresses, protocol, ports where available, and packet length.

• Calculate packet counts, byte totals, average packet size, protocol counts, top source IPs, and destination ports.

• Apply basic metadata-based alert rules.

• Export structured CSV reports and visualize records in a dashboard.

• Test important behavior using pytest.

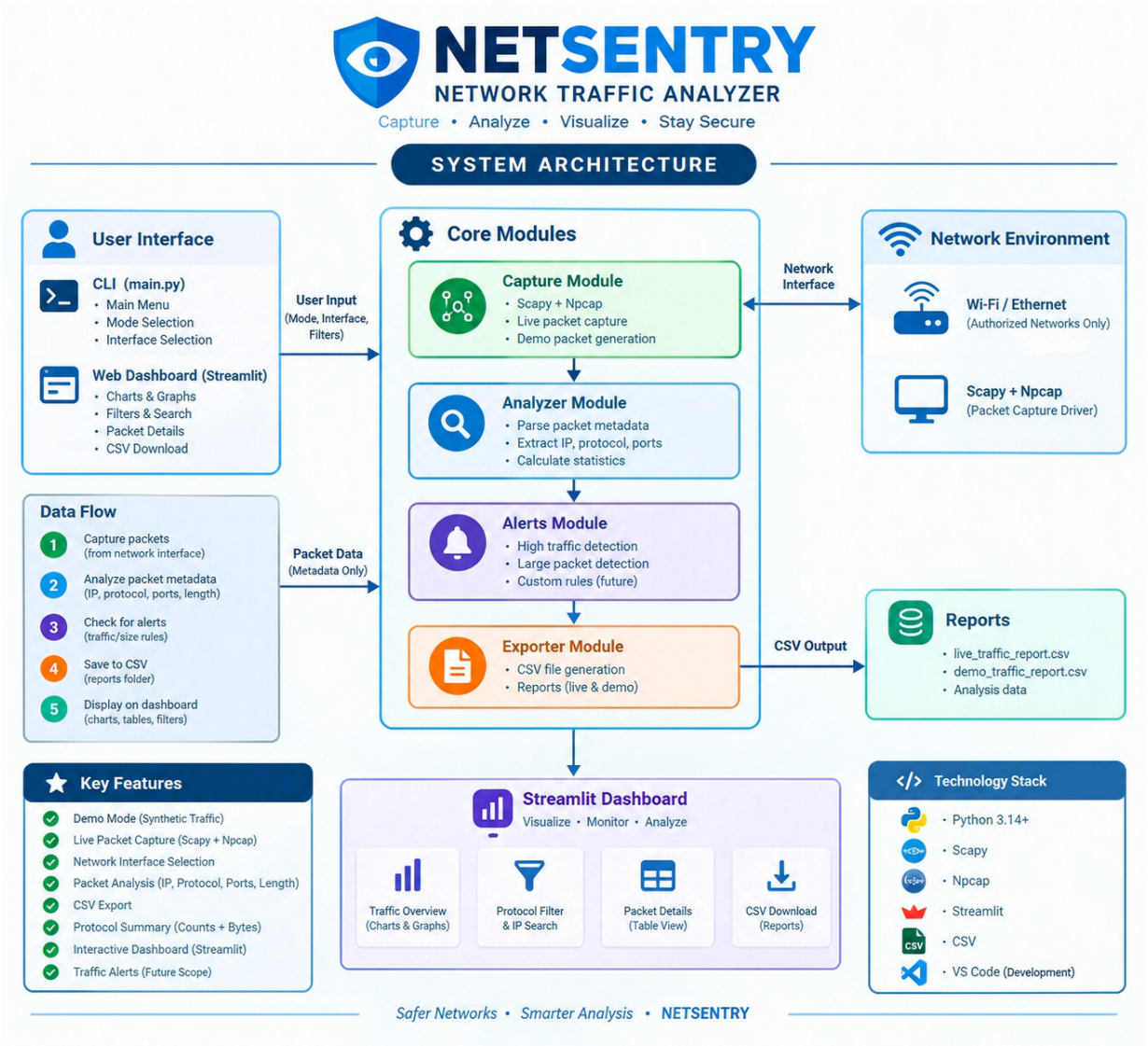
3. Technologies Used

Technology	Role in the project
Python	Core program logic, metadata processing, and statistics.
Scapy	Packet capture and packet-layer inspection.
Npcap	Windows packet-capture driver used by Scapy.
Streamlit	Interactive dashboard, charts, filters, packet table, and CSV download.
CSV	Portable output format for structured packet metadata.
pytest	Automated tests for analyzer, capture validation, statistics, and alerts.

4. System Architecture

At a high level, users interact through the command-line menu or the Streamlit dashboard. Packet input is generated synthetically in Demo Mode or captured from an authorized interface in Live Mode. The analyzer extracts metadata and calculates statistics. Alert logic evaluates configured conditions, while the exporter writes records to CSV. The dashboard visualizes the data using charts, filters, search, and a packet details table.

4. System Architecture — Diagram



The diagram is a conceptual overview. Module names and connections should be checked against the current source code before using it as an exact implementation map.

5. Features and Workflow

Demo Mode. Uses synthetic records so that dashboard and analysis behavior can be tested without capturing real network traffic.

Live Packet Capture. Uses Scapy and Npcap to capture a bounded number of packets from a selected interface. The tested run captured 10 packets within the configured capture window.

Packet Analysis. Extracts IP addresses, protocol, ports where available, timestamp, and packet length. The tool focuses on metadata and does not display packet payloads.

Statistics. Reports total packets, total bytes, average packet size, protocol counts, top source IPs, and destination ports.

Traffic Alerts. Includes simple heuristic rules for high packet counts from a source and unusually large packets. These rules are educational checks, not a complete intrusion detection system.

Dashboard. Provides traffic overview metrics, protocol distribution, packet-size charts, packet details, protocol filtering, IP search, and CSV download.

CSV Export. Saves structured packet metadata in report files for later inspection.

6. Project Structure

```
NetSentry/
■■■ main.py
■■■ app.py
■■■ network_sniffer/
■ ■■■ analyzer.py
■ ■■■ capture.py
■ ■■■ exporter.py
■ ■■■ alerts.py
■■■ reports/
■ ■■■ demo_traffic_report.csv
■ ■■■ live_traffic_report.csv
■■■ tests/
■ ■■■ test_analyzer.py
■ ■■■ test_capture.py
■ ■■■ test_alerts.py
■ ■■■ test_statistics.py
■■■ docs/
■■■ screenshots/
```

7. Testing and Results

The reported pytest run collected 15 tests and all 15 passed. The tests covered packet analyzer behavior, input validation for packet capture, capture behavior when no packets arrive, propagation of capture errors, statistics output, and traffic alert rules.

Area	Coverage	Result
Analyzer	TCP, UDP, ICMP, and non-IP packets.	PASS
Capture validation	Zero/excessive packet count and missing/unknown interface.	PASS
Capture behavior	No-packet case and capture error propagation.	PASS
Statistics	Statistics output for sample records.	PASS
Alerts	High traffic, large packet, normal traffic, and empty records.	PASS

Live capture result: 10 packets were captured: 8 TCP and 2 UDP. The total captured packet length was 810 bytes, and the average packet size was 81 bytes.

CSV validation: The live report contained 10 records, the expected seven columns (timestamp_utc, source_ip, destination_ip, protocol, source_port, destination_port, packet_length_bytes), and all packet lengths were positive.

Demo dashboard result: The displayed sample contained 3 packets, 3 protocol categories, and 96 total bytes. The dashboard screenshot showed charts, alerts, packet details, filters, and a CSV download control.

8. Security, Limitations, and Future Work

Security and privacy. Use NETSENTRY only on networks you own or are explicitly authorized to monitor. Live capture is bounded by packet-count and duration limits. The application is designed for metadata analysis and does not display payloads, passwords, or private messages.

Limitations. Results depend on the selected interface, available traffic, operating-system permissions, and Npcap configuration. A short capture may collect few or no packets. The current alert rules are simple heuristics that may produce false positives or miss suspicious activity. This tool is a learning project, not a replacement for a production monitoring or intrusion-detection system.

Future enhancements.

• Configurable thresholds and more alert-rule tests.

• Clearer mapping from Npcap interface paths to human-readable adapter names.

• Improved logging and error messages.

• Dashboard loading of saved live CSV reports.

• Additional protocol summaries and time-based charts.

• More integration and end-to-end tests.

9. Conclusion

NETSENTRY demonstrates an end-to-end introductory workflow for network traffic analysis, including demo and live packet input, metadata extraction, statistics, simple alert rules, visualization, and CSV reporting. The project includes automated tests and a GitHub repository with documentation and screenshots. It provides a practical foundation for learning packet analysis and network security concepts.

References

• Python documentation — https://docs.python.org/3/

• Scapy documentation — https://scapy.readthedocs.io/

• Streamlit documentation — https://docs.streamlit.io/

• pytest documentation — https://docs.pytest.org/

End of Report